

Deliverable 3: Architecture & Design

System design, key decisions, what breaks at Nucor scale

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

Architecture & Design

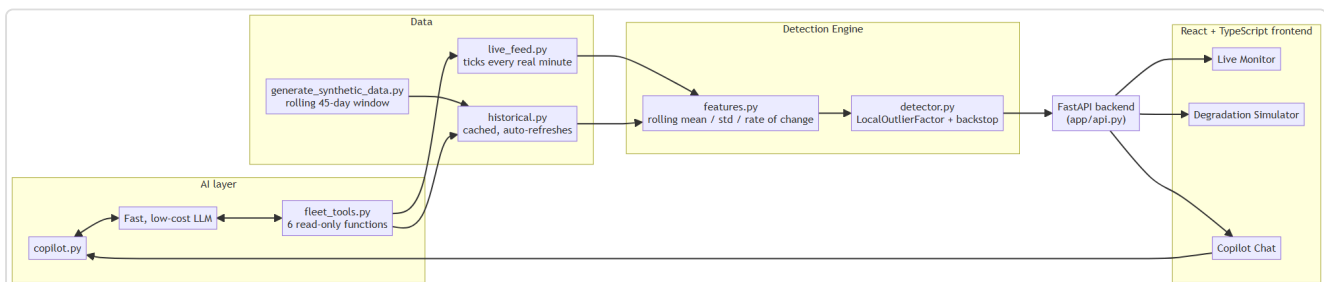
The problem, sharpened

The prompt as given is broad: help a shift supervisor prevent, anticipate, or react faster to unplanned downtime. That was narrowed hard, on purpose, instead of building a generic downtime dashboard:

- **User:** a shift supervisor on a hot mill line, specifically. Someone watching live operation and deciding, in the moment, whether to intervene, not a reliability engineer doing offline analysis.
- **Failure mode:** roll stand bearing degradation. One well understood mechanism with a real, defensible precursor signal, not every possible downtime cause.
- **The decision it speeds up:** should this stand be flagged for inspection before it fails on this shift. A concrete action a supervisor can actually take.

This does not predict all downtime causes, does not do automated shutdown (it flags and explains, a human decides), and does not run at production streaming scale. Those are named limits, not gaps papered over.

System design



Synthetic sensor data (or a real historian feed in production) flows through a feature pipeline (rolling mean, standard deviation, rate of change per signal) into a LocalOutlierFactor detector trained only on normal windows, backed by an independent deviation backstop for the model's own blind spot on sustained failures. The FastAPI backend serves that scored data to a React frontend with three surfaces: a live monitor, a hands on degradation simulator, and an AI copilot. The copilot calls six narrow, read only functions into the same live data instead of a vector database, since the entire fleet's current state easily fits in a single prompt at this scale.

What was considered and rejected

A supervised deep model instead of an unsupervised detector. Rejected: real predictive maintenance rarely has enough labeled failures to train something that data hungry, and an unsupervised detector only needs normal data, which is plentiful.

Picking a model by reasoning alone. This was a real gap in an early pass. Closed by an actual bake off (`notebooks/model_comparison.ipynb`) against IsolationForest, OneClassSVM, and EllipticEnvelope on identical features and metrics. LocalOutlierFactor won outright, zero false alarms versus roughly 0.8% for the runner up, and about ten times the warning time.

LocalOutlierFactor's local density blind spot. The same property that catches subtle onset early also means a sustained failure's own recent minutes eventually look normal to each other, and the score quietly drops. Fixed with an independent backstop keyed to a stable, whole training period baseline instead of a moving window, which also surfaced and fixed a real labeling bug in how clean normal data was defined.

Real time streaming versus batch. Batch scoring for training and evaluation, but the live console genuinely ticks in real time on top of it (`app/live_feed.py`). Still not production scale streaming, one in process background task, not a message bus.

Tool use over vector search for the AI copilot. Embeddings solve a corpus too big to fit in context. Six stands' current data does not need that; six real function calls into live data give a grounded answer with less machinery and no invented numbers.

Streamlit, then a real frontend. Streamlit validated the detection approach in an afternoon. Once the numbers were real, the interface was rebuilt properly in React and TypeScript with an actual API boundary, since a shift supervisor's tool needs to feel trustworthy, not like a script re-running top to bottom on every click.

Where this breaks at Nucor scale

- **One global threshold across all 6 stands,** not per-stand or per-mill calibration. Real stands differ in age, load, and calibration.
- **One consistent 5 signal schema assumed.** A real multi-division steelmaker almost certainly has different sensor vendors and tag naming per site.
- **No retraining cadence or drift monitoring.** Normal operation drifts; a model trained once and never revisited quietly gets worse.
- **Alert fatigue.** Even a low false alarm rate compounds across every stand, every mill, every shift. The single biggest realistic adoption risk, covered in the security and risk deliverable.
- **Assumes a supervisor checks a dashboard.** A real deployment needs to land inside whatever they already watch, not be one more screen to remember.

How this actually gets adopted

A model being accurate on paper does not mean a supervisor trusts it on day one. Turning alerts on for the whole floor immediately is the wrong first move: the first false alarm in week one costs more trust than ten correct alerts earn back, and a tool the crew has learned to ignore is worse than no tool at all. The real rollout is to run it silently for a few weeks against a crew that already trusts the process, let them check a handful of real flags against what they already know, and only then let it start paging people. That is a change management problem, not a modeling problem, and no amount of extra model accuracy fixes it on its own.

Full detail behind every decision above, including the bugs found getting here, is in the AI partnership log and `docs/architecture.md` in the repository.